Great choices. These three templates will cover your most critical workflows as a ServiceNow Consultant: reactive incidents (PIR), proactive deployments (Release Notes), and ongoing communication (Weekly Status).

Since the `docx` helper functions (`para`, `h1`, `h2`, `hdrCell`, `dataCell`, `statusCell`) from your original code are perfectly reusable, I have focused the JavaScript implementations below strictly on the extraction and table-building logic. You can drop these directly into your `main.js` file alongside your existing generator.

---

1. Major Incident Report (PIR)

This structure captures the timeline of the outage, the technical root cause, and the follow-up tasks required to prevent recurrence.

JSON Structure

```
{
  "incident_number": "INC0923847",
  "priority": "P1 - Critical",
  "short_description": "Database cluster failover failure in US-East",
  "assignment_group": "Database Admins",
  "timeline_events": [
    { "time": "08:15 AM", "event": "Alert Triggered", "description": "High CPU
utilization on Node 1" },
    { "time": "08:30 AM", "event": "Bridge Opened", "description": "Major Incident
bridge engaged" },
    { "time": "09:45 AM", "event": "Resolution", "description": "Manual failover
completed to Node 2" }
  ],
  "root_cause_analysis": [
    { "component": "Automated Failover Script", "issue": "Timeout threshold too
low", "fix": "Increased threshold from 30s to 120s" }
  ],
  "preventive_actions": [
    { "action": "Audit all cluster timeout settings", "owner": "DBA Team",
"status": "In Progress" },
    { "action": "Update runbook for manual failover", "owner": "Major Incident
Management", "status": "Pending" }
  ]
}
```

JavaScript Implementation

```
async function generatePIR(data) {
    const { Document, Packer, Paragraph, TextRun, Table, TableRow, TableCell,
WidthType, ShadingType, BorderStyle } = getDocx();
    const C = docxColors();

    // [Insert your existing helper functions here: bdr, borders, para, h1, h2,
```

```
hdrCell, dataCell, statusCell]

    const headerChildren = [
        h1('Post-Incident Review: ' + safeStr(data.incident_number)),
        para('Priority: ' + safeStr(data.priority), true, C.NAVY, 20),
        para('Description: ' + safeStr(data.short_description), false, C.GRAY,
18),
        para('Assignment Group: ' + safeStr(data.assignment_group), false, C.GRAY,
18),
        para('')
    ];

    if (data.timeline_events?.length > 0) {
        headerChildren.push(h2('Incident Timeline'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [1500, 2500, 5360],
            rows: [
                new TableRow({ children: [hdrCell('Time', 1500), hdrCell('Event',
2500), hdrCell('Description', 5360)] }),
                ...data.timeline_events.map((t, i) => new TableRow({
                    children: [
                        dataCell(t.time || '', 1500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(t.event || '', 2500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(t.description || '', 5360, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        }));
        headerChildren.push(para(''));
    }

    if (data.root_cause_analysis?.length > 0) {
        headerChildren.push(h2('Root Cause Analysis'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [2000, 3680, 3680],
            rows: [
                new TableRow({ children: [hdrCell('Component', 2000),
hdrCell('Issue', 3680), hdrCell('Fix Applied', 3680)] }),
                ...data.root_cause_analysis.map((r, i) => new TableRow({
                    children: [
                        dataCell(r.component || '', 2000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(r.issue || '', 3680, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(r.fix || '', 3680, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
```

```
        }));
        headerChildren.push(para(''));
    }

    if (data.preventive_actions?.length > 0) {
        headerChildren.push(h2('Preventive Actions'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [5000, 2560, 1800],
            rows: [
                new TableRow({ children: [hdrCell('Action', 5000),
hdrCell('Owner', 2560), hdrCell('Status', 1800)] }),
                ...data.preventive_actions.map((p, i) => new TableRow({
                    children: [
                        dataCell(p.action || '', 5000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(p.owner || '', 2560, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        statusCell(p.status || '', 1800) // Reusing your color-
coded status cell
                    ]
                }))
            ]
        }));
    }

    const doc = new Document({ sections: [{ properties: {}, children:
headerChildren }] });
    return { blob: await Packer.toBlob(doc), filename:
`${data.incident_number}_PIR.docx` };
}
```

---

2. ServiceNow Release & Deployment Notes

This structure breaks down an update set or application deployment, highlighting new features, bug fixes, and manual steps for CAB review.

JSON Structure

```
{
  "release_version": "v2.4.1",
  "deployment_date": "April 28, 2024",
  "application": "ITSM - Incident Management",
  "enhancements": [
    { "feature": "New VIP Flag UI", "description": "Added glowing indicator for
VIP callers on Incident form", "impact": "Low" }
  ],
  "defect_fixes": [
    { "ticket": "PRB004592", "issue": "Category field not resetting on subcategory
change", "resolution": "Updated OnChange Client Script" },
```

```
    { "ticket": "INC099123", "issue": "Email notifications double triggering",
"resolution": "Modified BR conditions" }
  ],
  "deployment_steps": [
    { "step": "Pre-Deployment", "description": "Backup sys_choice table XML",
"status": "Completed" },
    { "step": "Deployment", "description": "Commit Update Set: ITSM_Sprint_44",
"status": "Pending" },
    { "step": "Post-Deployment", "description": "Run fix script:
update_vip_flags.js", "status": "Pending" }
  ]
}
```

JavaScript Implementation

```
async function generateReleaseNotes(data) {
    const { Document, Packer, Paragraph, TextRun, Table, TableRow, TableCell,
WidthType, ShadingType, BorderStyle } = getDocx();
    const C = docxColors();

    // [Insert your existing helper functions here]

    const headerChildren = [
        h1('Release Notes: ' + safeStr(data.application)),
        para('Version: ' + safeStr(data.release_version), true, C.NAVY, 20),
        para('Target Deployment Date: ' + safeStr(data.deployment_date), false,
C.GRAY, 18),
        para('')
    ];

    if (data.enhancements?.length > 0) {
        headerChildren.push(h2('Enhancements & New Features'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [2500, 5000, 1860],
            rows: [
                new TableRow({ children: [hdrCell('Feature', 2500),
hdrCell('Description', 5000), hdrCell('Impact', 1860)] }),
                ...data.enhancements.map((e, i) => new TableRow({
                    children: [
                        dataCell(e.feature || '', 2500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(e.description || '', 5000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(e.impact || '', 1860, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        }));
        headerChildren.push(para(''));
```

```
    }

    if (data.defect_fixes?.length > 0) {
        headerChildren.push(h2('Defect Fixes'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [1500, 3930, 3930],
            rows: [
                new TableRow({ children: [hdrCell('Ticket', 1500),
hdrCell('Issue', 3930), hdrCell('Resolution', 3930)] }),
                ...data.defect_fixes.map((d, i) => new TableRow({
                    children: [
                        dataCell(d.ticket || '', 1500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(d.issue || '', 3930, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(d.resolution || '', 3930, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        }));
        headerChildren.push(para(''));
    }

    if (data.deployment_steps?.length > 0) {
        headerChildren.push(h2('Deployment Runbook'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [2000, 5560, 1800],
            rows: [
                new TableRow({ children: [hdrCell('Step Type', 2000),
hdrCell('Description', 5560), hdrCell('Status', 1800)] }),
                ...data.deployment_steps.map((s, i) => new TableRow({
                    children: [
                        dataCell(s.step || '', 2000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(s.description || '', 5560, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        statusCell(s.status || '', 1800)
                    ]
                }))
            ]
        }));
    }

    const doc = new Document({ sections: [{ properties: {}, children:
headerChildren }] });
    return { blob: await Packer.toBlob(doc), filename:
`Release_Notes_${data.release_version}.docx` };
}
```

4. Consultant Weekly Status & Sprint Report

This tracks your velocity, closed items, and any roadblocks requiring client intervention.

JSON Structure

```
{
  "consultant_name": "Jane Doe",
  "client_name": "Acme Corp",
  "week_ending": "April 26, 2024",
  "sprint_metrics": [
    { "metric": "Story Points", "planned": "35", "actual": "32", "status": "On
Track" },
    { "metric": "Hours Logged", "planned": "40", "actual": "42", "status":
"Completed" }
  ],
  "completed_tickets": [
    { "ticket": "STRY001928", "type": "Story", "description": "Build Inbound Email
Action for HR Cases" },
    { "ticket": "INC092837", "type": "Defect", "description": "Fix ACLs on
cmdb_ci_server table" }
  ],
  "current_blockers": [
    { "blocker": "Waiting on API credentials for Workday integration", "impact":
"High", "dependency": "Client Security Team" }
  ]
}
```

JavaScript Implementation

```
async function generateWeeklyStatus(data) {
    const { Document, Packer, Paragraph, TextRun, Table, TableRow, TableCell,
WidthType, ShadingType, BorderStyle } = getDocx();
    const C = docxColors();

    // [Insert your existing helper functions here]

    const headerChildren = [
        h1('Weekly Status Report'),
        para('Consultant: ' + safeStr(data.consultant_name), false, C.GRAY, 18),
        para('Client: ' + safeStr(data.client_name), false, C.GRAY, 18),
        para('Week Ending: ' + safeStr(data.week_ending), true, C.NAVY, 18),
        para('')
    ];

    if (data.sprint_metrics?.length > 0) {
        headerChildren.push(h2('Sprint Velocity'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [3000, 1500, 1500, 3360],
```

```
            rows: [
                new TableRow({ children: [hdrCell('Metric', 3000),
hdrCell('Planned', 1500), hdrCell('Actual', 1500), hdrCell('Status', 3360)] }),
                ...data.sprint_metrics.map((m, i) => new TableRow({
                    children: [
                        dataCell(m.metric || '', 3000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(m.planned || '', 1500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(m.actual || '', 1500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        statusCell(m.status || '', 3360)
                    ]
                }))
            ]
        }));
        headerChildren.push(para(''));
    }

    if (data.completed_tickets?.length > 0) {
        headerChildren.push(h2('Items Completed This Week'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [1500, 1500, 6360],
            rows: [
                new TableRow({ children: [hdrCell('Ticket', 1500), hdrCell('Type',
1500), hdrCell('Description', 6360)] }),
                ...data.completed_tickets.map((c, i) => new TableRow({
                    children: [
                        dataCell(c.ticket || '', 1500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(c.type || '', 1500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(c.description || '', 6360, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        }));
        headerChildren.push(para(''));
    }

    if (data.current_blockers?.length > 0) {
        headerChildren.push(h2('Risks & Blockers'));
        headerChildren.push(new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [4000, 1500, 3860],
            rows: [
                new TableRow({ children: [hdrCell('Blocker', 4000),
hdrCell('Impact', 1500), hdrCell('Dependency', 3860)] }),
                ...data.current_blockers.map((b, i) => new TableRow({
                    children: [
                        dataCell(b.blocker || '', 4000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
```

```
                        statusCell(b.impact || '', 1500), // Leveraging status
color logic
                        dataCell(b.dependency || '', 3860, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        }));
    }

    const doc = new Document({ sections: [{ properties: {}, children:
headerChildren }] });
    return { blob: await Packer.toBlob(doc), filename:
`Weekly_Status_${data.week_ending.replace(/[^a-z0-9]/gi, '_')}.docx` };
}
```